

Lab Procedure for Simulink

Sensor Fusion

Setup

1. It is recommended that you review [Lab 2 – Application Guide](#) before starting this lab.
2. Turn on the QBot Platform by pressing the power button once. To ensure the robot is ready for the lab, check the following conditions.
 - a. The LEDs on the robot base should be solid red.
 - b. The LCD should display the battery level. It is recommended that the battery level is over 12.5V.
 - c. The Logitech F710 joystick's wireless receiver is connected to the QBot Platform. Before use, **ensure the switch on top is in the X position and that the LED next to the Mode button is off.**
 - d. Make sure your computer is connected to the same network that the QBot Platform is on. If using the provided router, the network should be Quanser_UVS-5G.
 - e. Test connectivity to the QBot. Replace `192.168.2.x` with the IP address of the QBot displayed on the robot's LCD, enter the following command in your local computer terminal and hit enter: `ping 192.168.2.x`.
3. Deploy and run `qbot_platform_driver_physical` on QBot Platform:
 - a. Right click on `qbot_platform_driver_physical.rt-linux_qbot_platform`, select "Show more options", then select "Run on target".
 - b. Change Target URI to: `tcpip://192.168.2.x:17000`, replacing `192.168.2.x` with the IP address of the QBot.
 - c. Change Model Arguments to `-d /tmp -uri tcpip://192.168.2.x:17099`, replacing `192.168.2.x` with the IP address of the QBot.
 - d. Click Run.
 - e. The QBot Platform LEDs should pulse white if the driver is deployed and running successfully.
4. Open the Simulink Model `sensor_fusion.slx`, as shown in Figure 1. Configure the model so that it can be deployed to the QBot Platform:

- Open Hardware Settings  under the Hardware tab in your model.
- Expand and browse to **Code Generation > Interface**.
- Change the MEX-file arguments to the following string including single quotes,

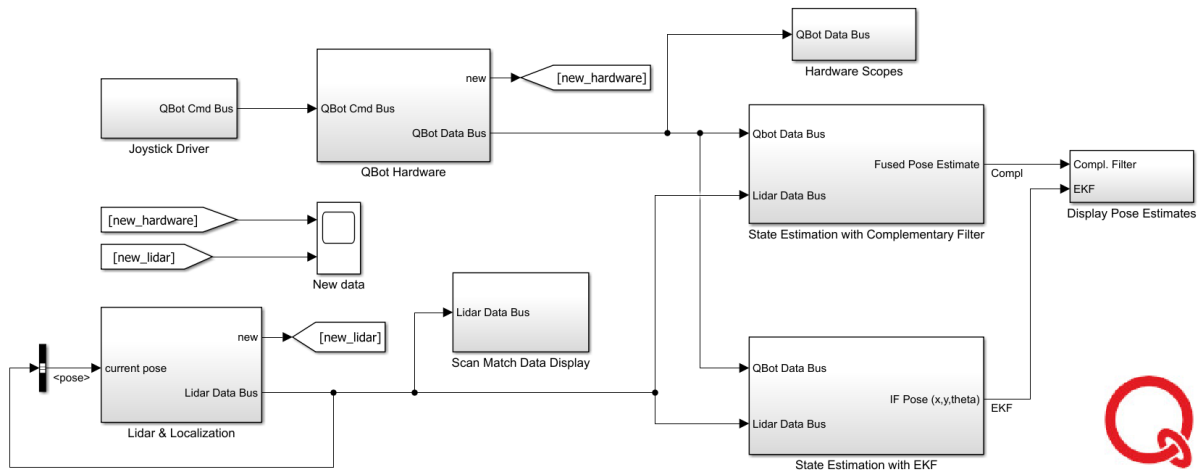


Figure 1: Sensor Fusion Simulink Model

1. This lab builds on **Lab 1: Localization** in Skills Progression 2. Ensure that the relevant sections of the model are completed with your code from Lab 1:
 - a. Open the **LiDAR & Localization** subsystem, then open the **Subsample and Correct LiDAR** subsystem. Open and complete the function block **correct_lidar**.
 - b. Return to the **LiDAR & Localization** subsystem and complete the connections between the subsystems.

2. In this lab you will explore sensor fusion techniques to localize the QBot Platform. In this section, you will construct a complementary filter to fuse two signals together to estimate the body forward speed of QBot Platform. You will start with estimating forward speed using the accelerometer signal.

To drive the QBot around using the joystick controller, press and hold the left button (LB) to arm the QBot, and keep LB pressed as you drive the QBot using the left and right joysticks. Note that this lab does not include obstacle detection functionality. To stop the QBot, disarm it by releasing LB. The joystick controls are as follows:

- a. Left joystick: Move left or right to turn the QBot body
- b. Right joystick: Move up or down to drive the QBot forward or back
- c. A button: (optional) Drive the QBot in a circle by applying a constant forward and turning speed command.

Run the model. Drive the QBot forward while observing the Acceleration scope. Identify the acceleration axis that corresponds to the forward movement direction? How do we estimate velocity from this signal? What is the expected signal value when the QBot is idle?

3. Press the right button (RB) to stop the code. Complete the connection shown in Figure 2 below to route the forward acceleration to an integrator block, which is connected to the **accelFwdSpd** block that routes the signal to the Forward Speed Estimate scope.

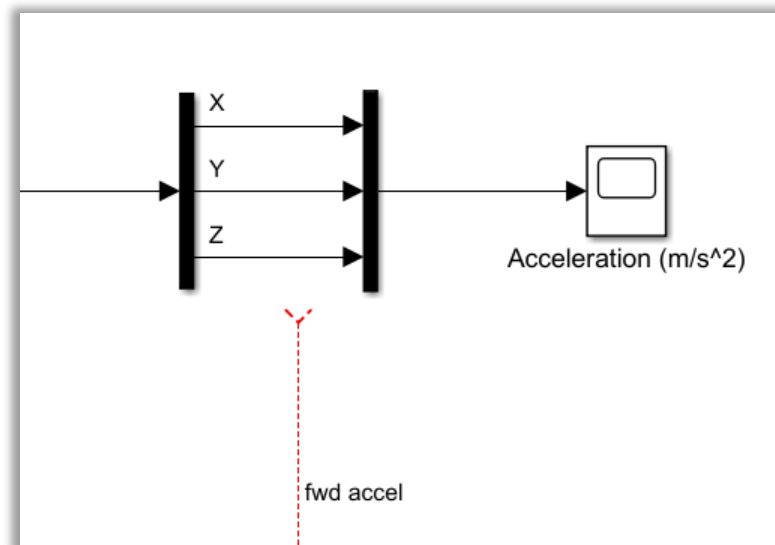


Figure 2: Connecting forward acceleration

4. Run the model. Without driving the QBot around, let the model run for at least 1 minute and observe the Forward Speed Estimate scope. Then, drive the QBot forward and backward. Release the left button (LB) to disarm the QBot and let it come to a stop. Is the forward speed estimated accurately when the QBot is idle and when it is moving? Stop the code.

- Next, you will use odometry to estimate the forward speed as done in Lab 1: Forward Kinematics from Skills Progression 1. Connect the Wheel Speeds signal to the **Demux** block that is connected to the **forKin** function block, which calculates the forward and turn speeds from the individual left and right wheel speeds. Connect its output, **vForward**, to the Forward Speed Estimate scope.
- Repeat Step 4 and observe both signals on the scope. What do you observe about the odometry-based forward speed estimate? How does this estimate compare to the accelerometer-based estimate?
- Now you will fuse the estimates from the individual sensors by correcting for the observed error in the accelerometer-based estimate using a proportional controller with the odometry-based estimate as the reference. In the Complementary Filter section, complete the connections shown in Figure 3: connect the output of the integrator to the + input of the first sum block, then apply a gain of K_p to this output before it is subtracted from the original accelerometer measurement. Set the value of K_p to 0.1.

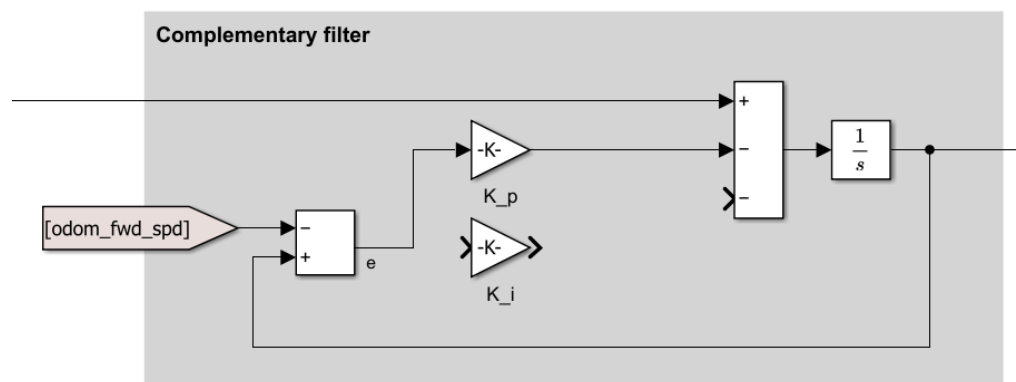


Figure 3: Proportional controller implementation

- Run the model. In the Forward Speed Estimate scope, enable the **fused forward speed estimate** signal by clicking the signal name on the legend, shown in Figure 4 below. While observing the Forward Speed Estimate scope, let the QBot idle for 30 seconds and then drive the QBot around. Compare the fused estimate to the estimates from the individual sensors. Stop the code.

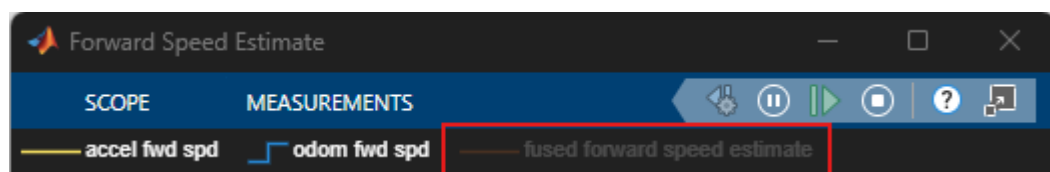


Figure 4: Enable the fused forward speed estimate from the scope legend

- Run the model. Observe the fused estimate on the scope as you set K_p to different values, including 0.5, 1, 5, and 10. How does increasing K_p affect the fused estimate?

10. Complete the implementation of the complementary filter by adding an integrator to the feedback controller as shown in Figure 5 below: Connect the output of the calculate error block to the gain K_i . Place an integrator block between the output of the K_i gain and the $-$ input of the sum block.

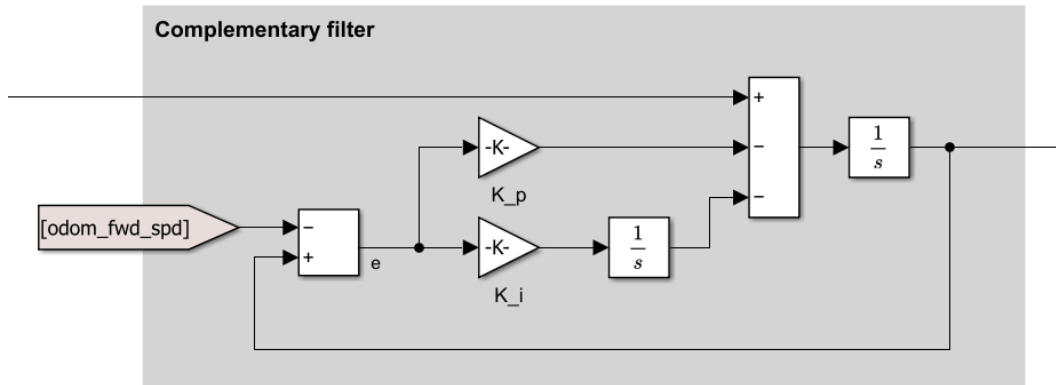


Figure 5: Integral controller implementation

11. Set the value of K_p to 1 and set the value of K_i to 0.05. Run the model and observe the signals on the scope as you increase and decrease K_i . How does the value of K_i affect the fused estimate?
12. Tune the values of K_p and K_i until the fused forward speed accurately tracks the correction signal with minimal noise. A sample tuned signal is shown in Figure 6.



Figure 6: Fused forward speed estimate with tuned K_p and K_i gains

13. Complete the implementations of the other two complementary filters. Start with the Turn Speed Complementary Filter. The rate input to the subsystem is the rate of the turn speed measured by the gyroscope. The correction signal is the body turn speed calculated from the measured wheel speeds.
14. You now have fused estimates of the QBot's forward speed and turn speed. These are converted to speeds along the X and Y directions and combined into a 3-element vector signal [x speed (m/s), y speed (m/s), omega (angular velocity) (rad/s)]. The Pose Complementary Filter uses this as the rate input and estimates the current position of the QBot using the position estimate from scan match as the correction input, which is a 3-element vector signal [x position (m), y position (m), theta].
15. Tune the K_p and K_i gains for the Pose Complementary filter. Run your code and take a screenshot of the final pose estimates as the QBot idles and as it is driving in a circle.

Comparison to Kalman Filter

16. Now, we will test an Extended Kalman Filter (EKF) for pose estimation using the same sensors and compare this sensor fusion technique to the complementary filter. In the main model [sensor_fusion.slx](#), right-click and uncomment the **Sensor Fusion with EKF** subsystem, which contains a completed implementation of an EKF. The EKF fuses new measurements into an estimate using an optimal weight that minimizes the estimate's uncertainty. Unlike the complementary filter, the EKF does not require manual gain tuning, since the weights are calculated using the motion model and measurement uncertainties. The completed implementation of the EKF to estimate the position of the QBot can be viewed inside the **Sensor Fusion with EKF** subsystem.
17. Return to the main model, open the Display Pose Estimates subsystem, and then open the Pose Estimates scope. Run the code. Drive the QBot in a circle while observing the Pose Estimates Scope. How do the estimates from the EKF compare to the estimates from the complementary filter?
18. Stop the Simulink model when complete by pressing the right button (RB). Ensure that you save a copy of your completed files for review later.
19. Turn OFF the QBot by single pressing the power button. Do not hold it down. After shutdown, confirm that all the LEDs are off.